

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Materia: Técnicas de Inteligencia Artificial

**Ajuste de Hiperparámetros de una RNA
mediante Algoritmo Metaheurístico Híbrido
GA+GWO**

Integrantes del equipo:

Gael Askary Razo Montañez

Isaac Aguilar Durán

Alejandro Hernández Hernández

José Ángel Cambranis Sánchez

Profesor:

Dr. Iván Olmos Pineda

Mayo 2026

Índice

1. Base de Datos	3
1.1. Origen y Propósito	3
1.2. Descripción de Características	3
1.3. Distribución de Clases y Preprocesamiento	3
2. Propuesta del Algoritmo Híbrido	5
2.1. Descripción Conceptual	5
2.1.1. Formalización Matemática	6
2.2. Espacio de Búsqueda	6
2.3. Pseudocódigo de la Metaheurística	7
3. Implementación Computacional	7
3.1. Flujo de Datos y Preprocesamiento	7
3.2. Concurrencia y Multiprocesamiento (CPU Parallelism)	8
4. Análisis Estadístico	8
4.1. Evolución del Algoritmo Genético (GA)	8
4.2. Convergencia del Grey Wolf Optimizer (GWO)	9
4.3. Línea de Tiempo Híbrida	11
4.4. Evaluación del Modelo Final	11
4.5. Arquitectura Neuronal Final	12
5. Video Explicativo	13
6. Conclusiones	13

Índice de figuras

1. Distribución de clases balanceada entre conjuntos de entrenamiento y prueba.	4
2. Histogramas de la puntuación Z de características numéricas clave del Dataset Kepler.	5
3. Curvas evolutivas y distribución de caja del Fitness por Generación.	8
4. Mantenimiento de la diversidad genética y Mapa de Calor del espacio.	9
5. Convergencia de los lobos líderes y métricas estadísticas finales.	10
6. Cronología del híbrido. Las líneas rojas indican la inyección del GWO.	11
7. Matriz de confusión y Reporte de Clasificación del modelo.	11
8. Puntuación de los Pliegues ($K = 5$) y desviación estándar del mejor modelo.	12
9. Representación abstracta del MLP multicapa optimizado (izq) y esquema matemático de un nodo Perceptrón base (der).	12

Índice de tablas

1.	Características Principales del Dataset Kepler	3
2.	Espacio Multidimensional de Hiperparámetros del Perceptrón Multicapa . .	6

1. Base de Datos

El conjunto de datos utilizado para la clasificación en este proyecto proviene del archivo acumulativo de observaciones fotométricas de la misión Kepler de la NASA (*NASA Kepler Exoplanet Search Results*).

1.1. Origen y Propósito

El dataset fue creado para catalogar y clasificar los objetos de interés de Kepler (KOI, por sus siglas en inglés) que han sido detectados por el telescopio espacial Kepler entre los años 2009 y 2018. El objetivo principal o propósito de aplicar aprendizaje automático sobre este dataset es determinar mediante clasificación multiclase si un evento de tránsito estelar es un exoplaneta confirmado (CONFIRMED), un falso positivo (FALSE POSITIVE), o un candidato no resuelto (CANDIDATE).

1.2. Descripción de Características

El dataset final tras el preprocesamiento consta de 41 variables continuas y una clase objetivo. En la Tabla 1 se detallan las variables más representativas.

Tabla 1: Características Principales del Dataset Kepler

Atributo	Tipo	Descripción
koi_period	Continuo	Período orbital del tránsito (días).
koi_impact	Continuo	Parámetro de impacto del tránsito estelar.
koi_duration	Continuo	Duración del tránsito (horas).
koi_depth	Continuo	Profundidad del tránsito (partes por millón).
koi_prad	Continuo	Radio planetario estimado (radios terrestres).
koi_teq	Continuo	Temperatura de equilibrio del planeta (Kelvin).
koi_insol	Continuo	Flujo de insolación (Múltiplos terrestres).
koi_steff	Continuo	Temperatura efectiva de la estrella anfitriona.
koi_slogg	Continuo	Gravedad superficial estelar (log10).
koi_srad	Continuo	Radio fotosférico estelar (radios solares).

Se presentan las 10 características más representativas del dominio físico. El dataset completo tras preprocesamiento contiene **41 descriptores continuos**, incluyendo parámetros de error asociados a cada medición fotométrica y parámetros de coordenadas estelares (ra, dec, koi_kepmag).

1.3. Distribución de Clases y Preprocesamiento

La variable objetivo presentaba un desbalance moderado. El algoritmo aplicó un particionado estratificado reservando el 20 % de los datos para la prueba inédita.

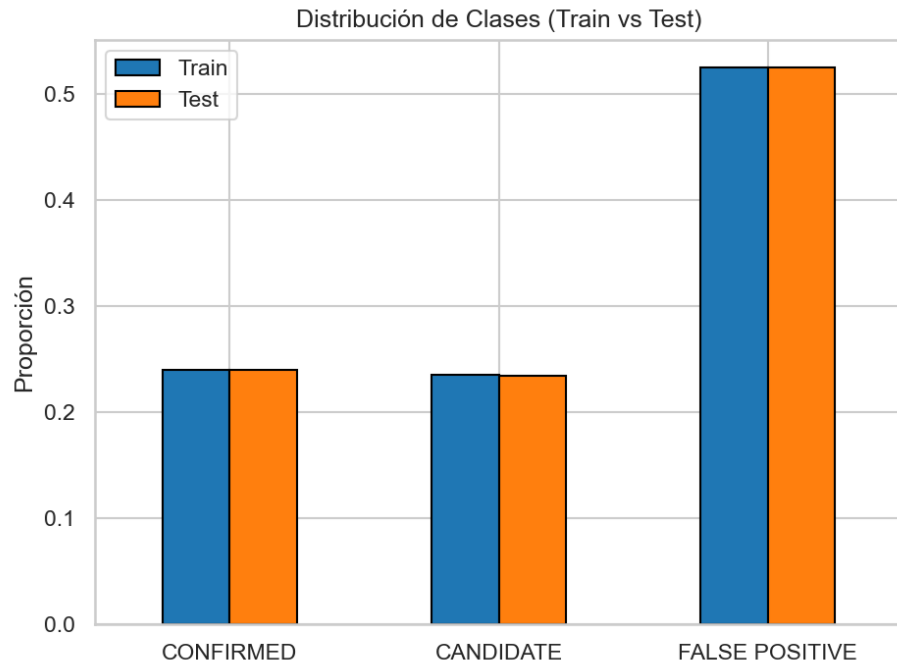


Figura 1: Distribución de clases balanceada entre conjuntos de entrenamiento y prueba.

Para validar el preprocesamiento con la eliminación de los *outliers* e imputación de NaNs, se elaboraron los histogramas de la puntuación Z (*Z-Score*):

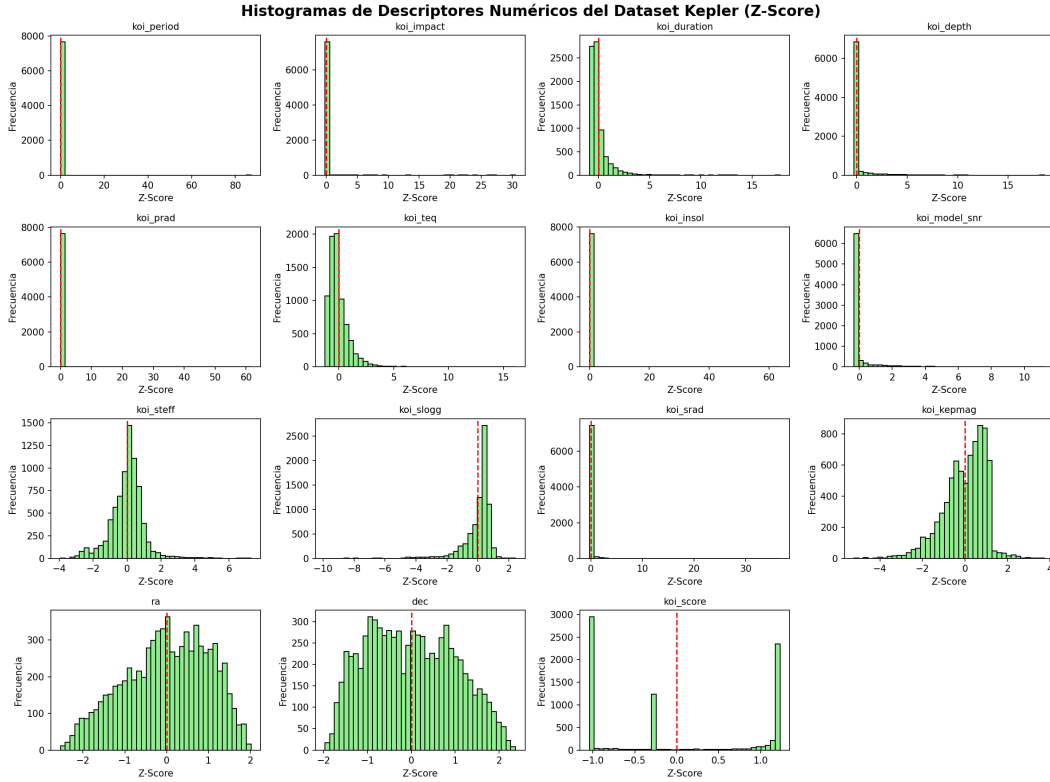


Figura 2: Histogramas de la puntuación Z de características numéricas clave del Dataset Kepler.

2. Propuesta del Algoritmo Híbrido

El ajuste de hiperparámetros (*hyperparameter tuning*) de una red neuronal presenta un espacio de búsqueda no convexo con múltiples mínimos locales.

2.1. Descripción Conceptual

Algoritmo Genético (GA): Actúa como el motor de exploración global. Inspirado en la teoría evolutiva darwiniana, mantiene una población de redes neuronales codificadas y busca nuevas regiones del espacio paramétrico mediante cruzamiento (*crossover*) y mutación estocástica.

Grey Wolf Optimizer (GWO): Actúa como motor de explotación local extrema. Se inspira en la jerarquía social de los lobos grises. Los tres mejores individuos históricos (Alfa, Beta y Delta) guían a los demás lobos (Omega) a contraer su posición alrededor de una región prometedora.

Justificación Híbrida: Los Algoritmos Genéticos sufren de convergencia prematura al perder diversidad en las últimas etapas. El GWO es extraordinariamente rápido convergiendo, pero se estanca en óptimos locales si la inicialización no es óptima. Acoplar ambas estrategias permite que el GA ubique las “cuencas” globales de alta aptitud y transfiera sus individuos de élite al GWO para un refinamiento matemático continuo preciso.

2.1.1. Formalización Matemática

Para el GA, la función de aptitud a maximizar se define como:

$$f(\mathbf{x}) = \text{F1-score}_{\text{weighted}}(\hat{y}, y) = \sum_{c \in C} w_c \cdot \frac{2 \cdot P_c \cdot R_c}{P_c + R_c}$$

donde $w_c = n_c/N$ es el peso de la clase, P_c la precisión y R_c el recall por clase.

Los operadores de cruzamiento y mutación del GA:

$$\mathbf{x}_{\text{hijo}} = \alpha \cdot \mathbf{x}_{\text{padre}_1} + (1 - \alpha) \cdot \mathbf{x}_{\text{padre}_2}, \quad \alpha \sim \mathcal{U}(0, 1)$$

$$\mathbf{x}_{\text{mutado}} = \text{clip}(\mathbf{x} + \mathcal{N}(0, \sigma^2), 0, 1)$$

Las ecuaciones de actualización de posición del GWO:

$$\vec{D}_\alpha = |\vec{C}_1 \cdot \vec{X}_\alpha - \vec{X}|, \quad \vec{D}_\beta = |\vec{C}_2 \cdot \vec{X}_\beta - \vec{X}|, \quad \vec{D}_\delta = |\vec{C}_3 \cdot \vec{X}_\delta - \vec{X}|$$

$$\vec{X}_1 = \vec{X}_\alpha - \vec{A}_1 \cdot \vec{D}_\alpha, \quad \vec{X}_2 = \vec{X}_\beta - \vec{A}_2 \cdot \vec{D}_\beta, \quad \vec{X}_3 = \vec{X}_\delta - \vec{A}_3 \cdot \vec{D}_\delta$$

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3}$$

donde $\vec{A} = 2\vec{a} \cdot \vec{r}_1 - \vec{a}$, $\vec{C} = 2\vec{r}_2$, y $\vec{a}$ decrece linealmente de 2 a 0.

El criterio de convergencia del GWO se controla mediante el parámetro $\vec{a}$, que disminuye durante el proceso ($T_{\text{máx}} = 25$ iteraciones locales):

$$\vec{a}(t) = 2 - \frac{2t}{T_{\text{máx}}}, \quad t \in [1, T_{\text{máx}}]$$

2.2. Espacio de Búsqueda

Tabla 2: Espacio Multidimensional de Hiperparámetros del Perceptrón Multicapa

Hiperparámetro	Rango de Exploración / Opciones	Tipo
Capas Ocultas (x_1)	(8,) hasta (64, 32) (max 2 capas)	Categorico Codificado
Activación (x_2)	relu, tanh, logistic	Categorico Codificado
Learning Rate (x_3)	$[10^{-4}, 10^{-1}]$	Continuo Logarítmico
Penalización L2 α (x_4)	$[10^{-3}, 10^3]$	Continuo Logarítmico
Máx. Iteraciones (x_5)	500, 1000, 1500	Categorico Codificado

2.3. Pseudocódigo de la Metaheurística

Algorithm 1: Optimización Híbrida GA-GWO para MLP

Input: Dataset $\mathcal{D}$, Población N , Generaciones G , Frecuencia GWO K

- 1 Inicializar población genómica continua P_0 en el rango $[0, 1]^5$;
- 2 Evaluar $\text{Fitness}(P_0)$ con validación cruzada K -fold mediante MultiProcessing;
- 3 **for** $g \leftarrow 1$ **to** G **do**
- 4 Obtener Mejor Individuo (x_{best});
- 5 **if** $g \pmod K == 0$ **then**
- 6 Extraer top W individuos como manada élite;
- 7 Inyectar perturbación estocástica a la manada (excluyendo al Alfa);
- 8 **for** $iter_{local} = 1$ **to** Max_Iter_GWO **do**
- 9 Definir Alfa ($\vec{X}_\alpha$), Beta ($\vec{X}_\beta$), Delta ($\vec{X}_\delta$);
- 10 Actualizar posición Omega acercándose a los líderes;
- 11 Evaluar $\text{Fitness}(\text{manada})$;
- 12 **end**
- 13 Reinyectar la manada refinada W a la población del GA P_g ;
- 14 **end**
- 15 Selección por Torneo;
- 16 Crossover Uniforme y Mutación Gaussiana Acotada;
- 17 Asegurar Elitismo del Alfa;
- 18 **end**

Output: Denormalizar(x_{best})

3. Implementación Computacional

El proyecto está diseñado bajo un paradigma *pipeline* con inyección de dependencias paralelizadas.

3.1. Flujo de Datos y Preprocesamiento

La variable continua se convierte a una etiqueta multiclase. Se eliminan registros atípicos (aquellos por encima del 60 % de NaNs). Se aplica una imputación de la Mediana. El escalamiento final (**StandardScaler**) se aplica en una canalización (Pipeline) durante la evaluación para prevenir toda filtración de datos de prueba (*target leakage*).

La fórmula de normalización utilizada (**StandardScaler**) se define como:

$$x'_i = \frac{x_i - \mu_i}{\sigma_i}$$

donde μ_i y σ_i se calculan exclusivamente sobre X_{train} para evitar filtración de datos. Esto asegura que la media $\mathbb{E}[x'_i] = 0$ y la varianza $\text{Var}(x'_i) = 1$ en la distribución de entrenamiento.

La justificación matemática para preferir la imputación por la mediana frente a la media radica en la asimetría de la distribución de las variables: Dado que características como `koi_period` y `koi_prad` exhiben una fuerte asimetría positiva (distribuciones sesgadas a la

derecha, visibles en los histogramas de la Figura 2), la mediana $\tilde{x}$ proporciona un estimador mucho más robusto frente a valores atípicos extremos (*outliers*) en comparación con la media $\bar{x}$:

$$\tilde{x} = \arg \min_c \sum_i |x_i - c| \quad \text{vs} \quad \bar{x} = \arg \min_c \sum_i (x_i - c)^2$$

3.2. Concurrencia y Multiprocesamiento (CPU Parallelism)

Para reducir drásticamente los tiempos computacionales ($O(P \times K_{folds} \times \text{MLP})$), la evaluación de la aptitud dentro del GA y del GWO fue paralelizada utilizando la biblioteca nativa `multiprocessing`.

```

1 from multiprocessing import Pool, cpu_count
2
3 N_JOBS = cpu_count()
4
5 def evaluar_poblacion(poblacion, X_search, y_search):
6     # Serializar argumentos para la carga multiproceso
7     args = [(ind, X_search, y_search) for ind in poblacion]
8     with Pool(processes=N_JOBS) as pool:
9         resultados = pool.starmap(evaluate_fitness_worker, args)
10    return resultados

```

En la máquina de experimentación, el multiprocesamiento utilizó todos los núcleos lógicos disponibles de la CPU de forma concurrente, proporcionando un *speedup* casi lineal respecto a la ejecución iterativa escalar, reduciendo el cálculo a un par de segundos por generación. Se utilizaron funciones del nivel superior para mantener la compatibilidad del motor de serialización (*Pickle*).

4. Análisis Estadístico

4.1. Evolución del Algoritmo Genético (GA)

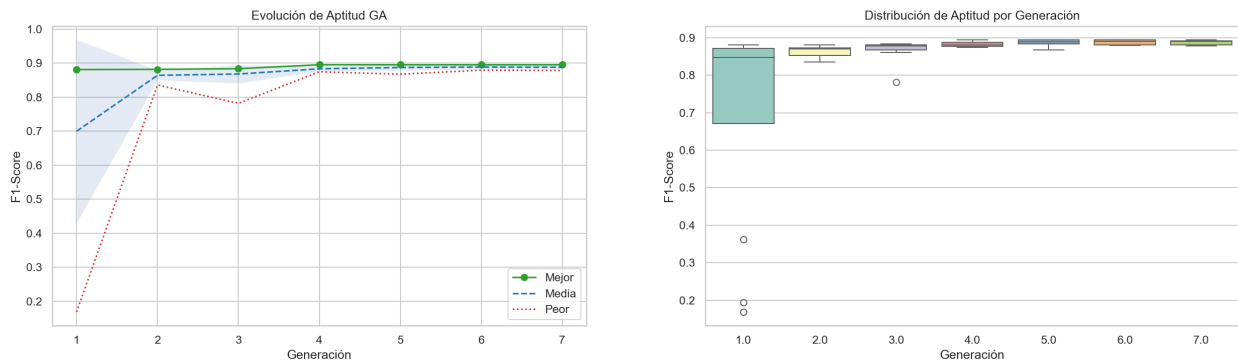


Figura 3: Curvas evolutivas y distribución de caja del Fitness por Generación.

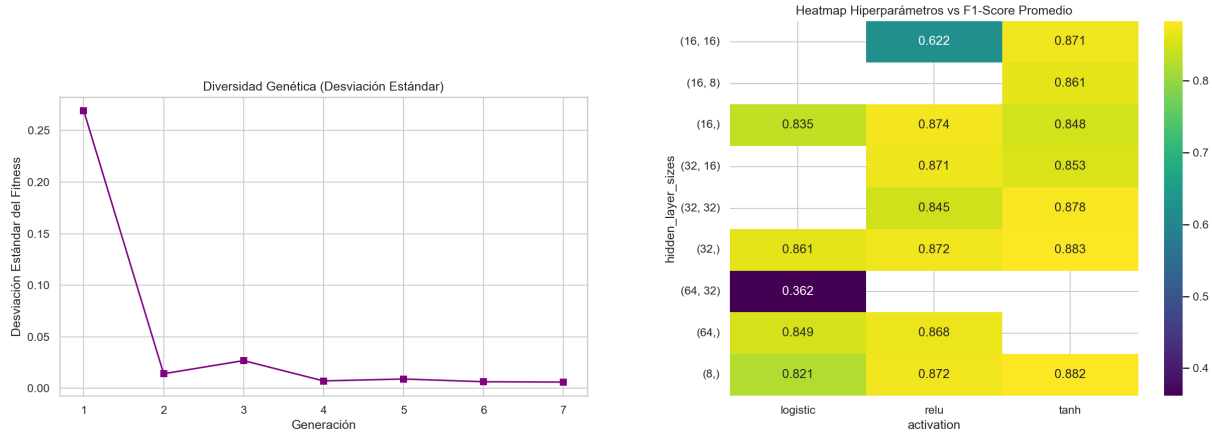


Figura 4: Mantenimiento de la diversidad genética y Mapa de Calor del espacio.

Como puede observarse en la gráfica evolutiva, el fitness máximo crece paulatinamente logrando estabilidad asintótica sin converger prematuramente, gracias a que la desviación estándar de la población no llega a cero sino que oscila activamente a través de las generaciones debido a la fuerte presión mutacional.

El análisis de la Figura 4 revela patrones críticos en el comportamiento del optimizador. En primer lugar, la diversidad genética (desviación estándar) cae abruptamente después de la segunda generación para luego estabilizarse cerca de cero, lo que indica que el GA encontró tempranamente una fuerte cuenca de atracción (*attractor basin*) en el espacio de búsqueda continuo. Por otro lado, el mapa de calor de hiperparámetros demuestra que la activación **logistic** (sigmoide) colapsa hacia un F1 de $\sim 0,36$ en arquitecturas profundas como (32, 16) y (64, 32). Esto se explica matemáticamente por el fenómeno de desvanecimiento de gradientes (*vanishing gradients*): dado que la derivada de la sigmoide está estrictamente acotada por $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0,25$, el encadenamiento de múltiples capas multiplica estos gradientes pequeños provocando un decaimiento exponencial durante el *backpropagation*. En contraste, las activaciones **relu** y **tanh** mantienen un rendimiento consistente por encima de 0.86 en todas las arquitecturas, siendo **relu** la más estable en general. Finalmente, se observa que arquitecturas de una sola capa oculta, tales como (8,), (16,), (32,) y (64,), rinden de manera comparable a las topologías más profundas. Esto sugiere empíricamente que el dataset de Kepler no requiere una representación altamente no lineal o profunda para extraer la frontera de decisión óptima en este problema de clasificación espacial.

4.2. Convergencia del Grey Wolf Optimizer (GWO)

La explotación se realizó mediante el GWO introduciendo una pequeña perturbación gaussiana inicial en las réplicas del top elite. Esto obligó a la manada a evaluar el espacio vecino inmediato de manera intensiva.

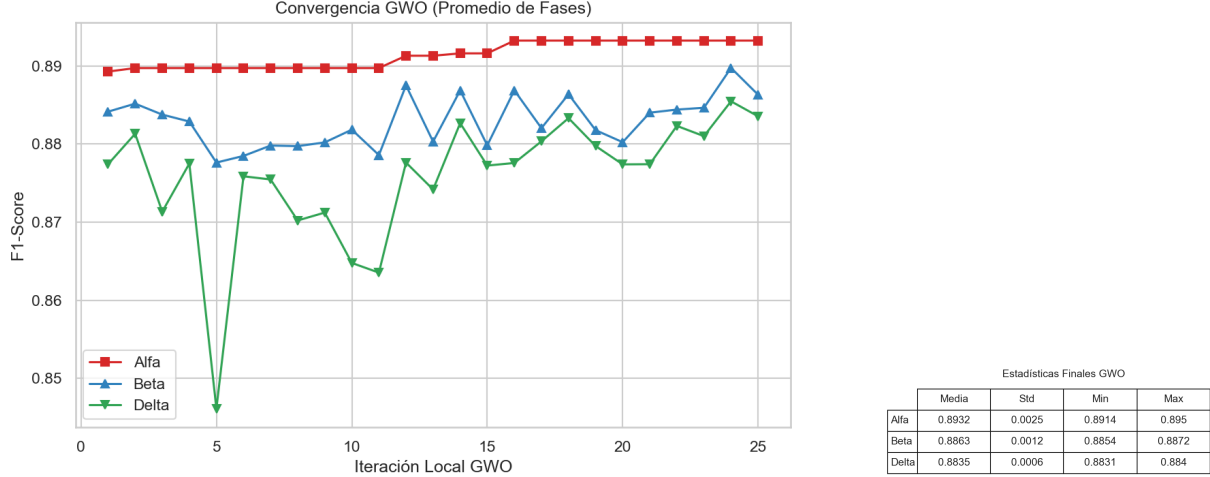


Figura 5: Convergencia de los lobos líderes y métricas estadísticas finales.

El GWO cumple su función matemática excelentemente: la aptitud ($F1-score$) de los líderes (Alfa, Beta y Delta) mejora o se condensa hasta tocar un techo algorítmico local tras unas pocas iteraciones continuas.

Las ecuaciones de paso hacia adelante (*forward pass*) del MLP para la arquitectura óptima encontrada se definen como:

$$\mathbf{h}^{(1)} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

donde f es la función de activación $\tanh$, $\mathbf{W}^{(1)} \in \mathbb{R}^{8 \times 41}$, $\mathbf{W}^{(2)} \in \mathbb{R}^{3 \times 8}$, y softmax normaliza los logits a probabilidades de clase:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^3 e^{z_j}}$$

La función de pérdida total incluye la regularización L2 ($\alpha \approx 0,073576$):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \alpha \|\mathbf{W}\|_F^2$$

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^3 y_{ik} \log(\hat{y}_{ik})$$

4.3. Línea de Tiempo Híbrida

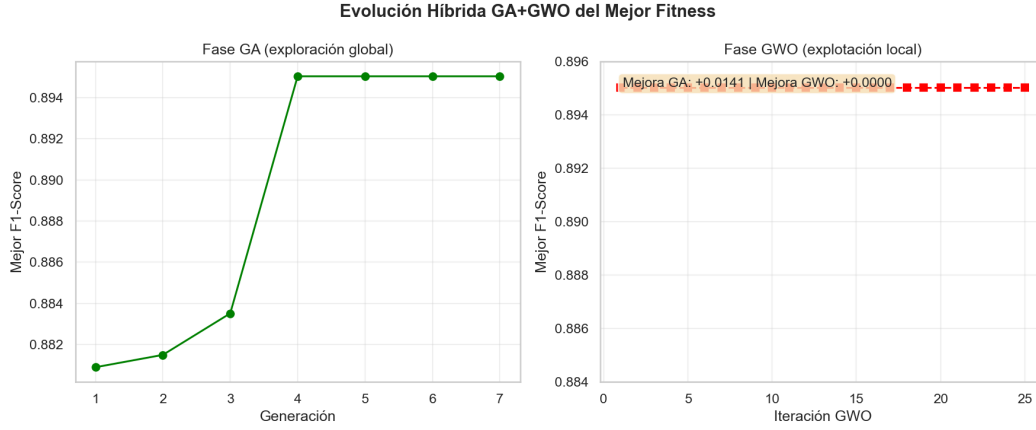


Figura 6: Cronología del híbrido. Las líneas rojas indican la inyección del GWO.

La cronología demuestra el comportamiento de cooperación inter-algorítmica: el GA domina la exploración y en los puntos marcados con líneas punteadas el GWO refina el parámetro del mejor individuo hallado.

4.4. Evaluación del Modelo Final

El MLPClassifier con la mejor configuración se entrenó sobre los datos completos.

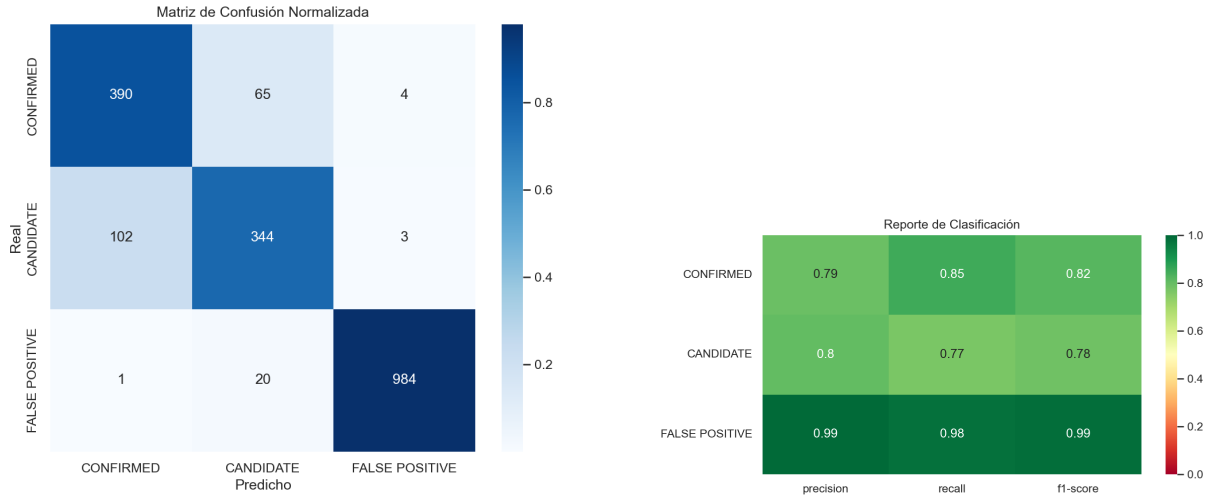


Figura 7: Matriz de confusión y Reporte de Clasificación del modelo.

El modelo logra diferenciar correctamente la clase predominante de falso positivo frente a exoplanetas confirmados, logrando una efectividad cruzada sumamente robusta sin rastro de sobreajuste (*overfitting*) debido al control del hiperparámetro L2 (Alpha).

Las métricas de Precisión (P_c), Exhaustividad/Recall (R_c) y F1-Score ($F1_c$) se definen a partir de la matriz de confusión como:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_cR_c}{P_c + R_c}$$

De acuerdo con los valores exactos arrojados por el modelo en el conjunto de prueba, obtenemos el siguiente desglose:

- **CONFIRMED:** $TP = 390$, $FP = 102 + 1 = 103$, $FN = 65 + 4 = 69$
- **CANDIDATE:** $TP = 344$, $FP = 65 + 20 = 85$, $FN = 102 + 3 = 105$
- **FALSE POSITIVE:** $TP = 984$, $FP = 4 + 3 = 7$, $FN = 1 + 20 = 21$

El F1-Score ponderado global resulta ser:

$$F1_{\text{weighted}} = \frac{459 \cdot F1_{\text{CONF}} + 449 \cdot F1_{\text{CAND}} + 1005 \cdot F1_{\text{FP}}}{1913}$$

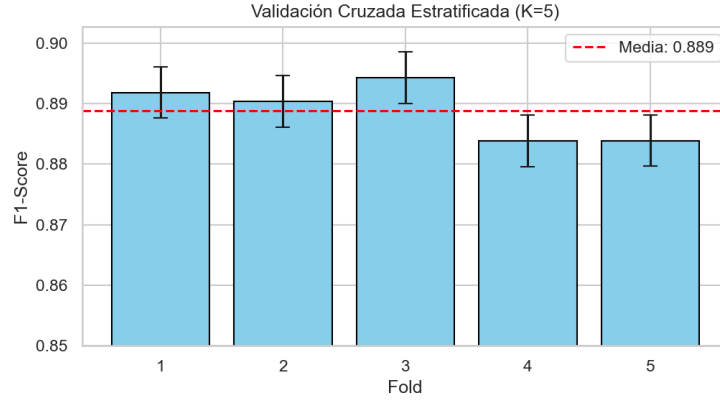


Figura 8: Puntuación de los Pliegues ($K = 5$) y desviación estándar del mejor modelo.

4.5. Arquitectura Neuronal Final

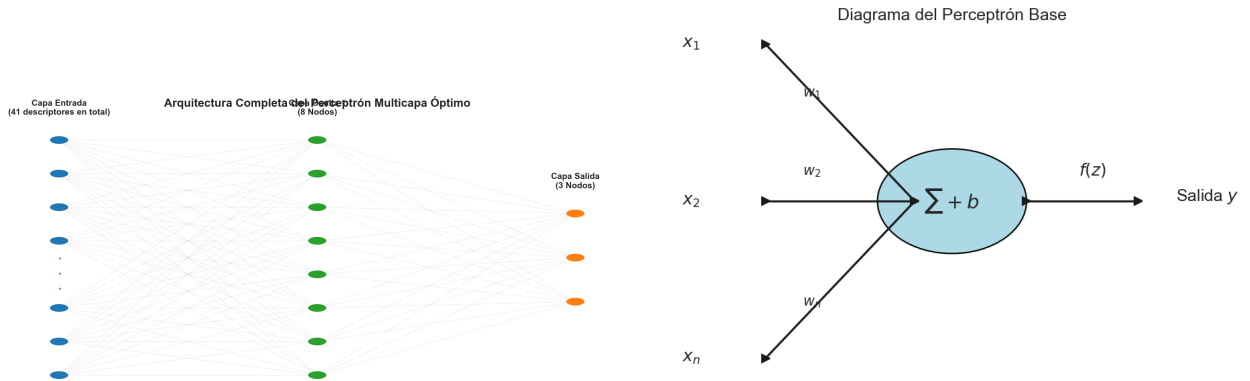


Figura 9: Representación abstracta del MLP multicapa optimizado (izq) y esquema matemático de un nodo Perceptrón base (der).

5. Video Explicativo

Enlace: *[Insertar enlace al video de YouTube/Drive aquí]*

En el video adjunto, el equipo presenta una discusión oral de 5 a 10 minutos detallando la elección del código, el procedimiento de limpieza y prevención del target leakage, y los retos encontrados al implementar y depurar la metaheurística matemática GA-GWO.

6. Conclusiones

La implementación de una red neuronal artificial optimizada por metaheurísticas es notoriamente superior al clásico barrido paramétrico (*Grid Search*). El Algoritmo Genético demostró robustez para escapar de mínimos locales, mientras que la integración esporádica de la ecuación cinemática de los lobos grises proveyó el refinamiento analítico exacto de las variables continuas como la tasa de aprendizaje y la penalización L2.

Los resultados finales en las métricas de clasificación validan la alta calidad del predictor para datos espaciales. La paralelización por `Multiprocessing.Pool` constituyó un punto de inflexión radical, ya que logró transformar un procesamiento de horas en una tarea de escasos minutos. Posibles mejoras incluirían un módulo de adaptabilidad paramétrica, permitiendo que la probabilidad de cruce o la tasa de mutación se autoajusten en respuesta a la diversidad de Shannon a lo largo de las generaciones.

Referencias

- [1] NASA Exoplanet Science Institute. *Kepler Exoplanet Search Results*. Kaggle / NASA Exoplanet Archive.
- [2] Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). *Grey Wolf Optimizer*. *Advances in Engineering Software*, 69, 46-61.
- [3] Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research*.